

Configuration File: Database LookUp.xml

```

,~/message/PATIENT_RESULT_GROUP/ORDER_OBSERVATION_GROUP/OBSERVATION_GROUP/rowELR_MESSAGE_OBR
OBX/OBX_3_2`
,~/message/PATIENT_RESULT_GROUP/ORDER_OBSERVATION_GROUP/OBSERVATION_GROUP/rowELR_MESSAGE_OBR
OBX/OBX_3_3`
,~/message/PATIENT_RESULT_GROUP/ORDER_OBSERVATION_GROUP/OBSERVATION_GROUP/rowELR_MESSAGE_OBR
OBX/OBX_3_4`
,~/message/PATIENT_RESULT_GROUP/ORDER_OBSERVATION_GROUP/OBSERVATION_GROUP/rowELR_MESSAGE_OBR
OBX/OBX_3_5`
,~/message/PATIENT_RESULT_GROUP/ORDER_OBSERVATION_GROUP/OBSERVATION_GROUP/rowELR_MESSAGE_OBR
OBX/OBX_3_6`
,~/message/PATIENT_RESULT_GROUP/ORDER_OBSERVATION_GROUP/OBSERVATION_GROUP/rowELR_MESSAGE_OBR
OBX/OBX_4`
,~/message/PATIENT_RESULT_GROUP/ORDER_OBSERVATION_GROUP/OBSERVATION_GROUP/rowELR_MESSAGE_OBR
OBX/OBX_5`
,~/message/PATIENT_RESULT_GROUP/ORDER_OBSERVATION_GROUP/OBSERVATION_GROUP/rowELR_MESSAGE_OBR
OBX/OBX_6_1`
,~/message/PATIENT_RESULT_GROUP/ORDER_OBSERVATION_GROUP/OBSERVATION_GROUP/rowELR_MESSAGE_OBR
OBX/OBX_6_2`
,~/message/PATIENT_RESULT_GROUP/ORDER_OBSERVATION_GROUP/OBSERVATION_GROUP/rowELR_MESSAGE_OBR
OBX/OBX_6_3`
,~/message/PATIENT_RESULT_GROUP/ORDER_OBSERVATION_GROUP/OBSERVATION_GROUP/rowELR_MESSAGE_OBR
OBX/OBX_7`
,~/message/PATIENT_RESULT_GROUP/ORDER_OBSERVATION_GROUP/OBSERVATION_GROUP/rowELR_MESSAGE_OBR
OBX/OBX_8`
,~/message/PATIENT_RESULT_GROUP/ORDER_OBSERVATION_GROUP/OBSERVATION_GROUP/rowELR_MESSAGE_OBR
OBX/OBX_9`
,~/message/PATIENT_RESULT_GROUP/ORDER_OBSERVATION_GROUP/OBSERVATION_GROUP/rowELR_MESSAGE_OBR
OBX/OBX_10`
,~/message/PATIENT_RESULT_GROUP/ORDER_OBSERVATION_GROUP/OBSERVATION_GROUP/rowELR_MESSAGE_OBR
OBX/OBX_11`
,~/message/PATIENT_RESULT_GROUP/ORDER_OBSERVATION_GROUP/OBSERVATION_GROUP/rowELR_MESSAGE_OBR
OBX/OBX_12`
,~/message/PATIENT_RESULT_GROUP/ORDER_OBSERVATION_GROUP/OBSERVATION_GROUP/rowELR_MESSAGE_OBR
OBX/OBX_13`
,~/message/PATIENT_RESULT_GROUP/ORDER_OBSERVATION_GROUP/OBSERVATION_GROUP/rowELR_MESSAGE_OBR
OBX/OBX_14`
,~/message/PATIENT_RESULT_GROUP/ORDER_OBSERVATION_GROUP/OBSERVATION_GROUP/rowELR_MESSAGE_OBR
OBX/OBX_14_CONVERTED`
,~/message/PATIENT_RESULT_GROUP/ORDER_OBSERVATION_GROUP/OBSERVATION_GROUP/rowELR_MESSAGE_OBR
OBX/OBX_15_1`
,~/message/PATIENT_RESULT_GROUP/ORDER_OBSERVATION_GROUP/OBSERVATION_GROUP/rowELR_MESSAGE_OBR
OBX/OBX_15_2`
,~/message/PATIENT_RESULT_GROUP/ORDER_OBSERVATION_GROUP/OBSERVATION_GROUP/rowELR_MESSAGE_OBR
OBX/OBX_15_3`
,~/message/PATIENT_RESULT_GROUP/ORDER_OBSERVATION_GROUP/OBSERVATION_GROUP/rowELR_MESSAGE_OBR
OBX/OBX_16`
,~/message/PATIENT_RESULT_GROUP/ORDER_OBSERVATION_GROUP/OBSERVATION_GROUP/rowELR_MESSAGE_OBR
OBX/OBX_17`
,~/message/PATIENT_RESULT_GROUP/ORDER_OBSERVATION_GROUP/OBSERVATION_GROUP/rowELR_MESSAGE_OBR
OBX/NTE_INCLUDED_FL`
,SYSDATETIME()
)</sql>

<statement id:type="compoundSqlStatement">
  <sql>SELECT \@ \@IDENTITY AS ELR_MESSAGE_OBX_ID</sql>
</statement>
</statement>
<statement name="Insert Into ELR_MESSAGE_OBX5 Table" id:type="executeSql" iterate
="PATIENT_RESULT_GROUP/ORDER_OBSERVATION_GROUP/OBSERVATION_GROUP/OBX5_GROUP/rowELR_MESSAGE

```

Configuration File: Database LookUp.xml

```

_OBX5" ide:summary="Insert data into the ELR_MESSAGE_OBX5 table. Many OBX-5 components are possible depending on the data type." ide:allowCompoundStatements="true" user:iterationRootMessagePath="message">
    <sql>INSERT INTO ELR_MESSAGE_OBX5 (
        ELR_MESSAGE_OBX_ID
        ,OBX_5_COMPONENT_NUMBER
        ,RELATED_VALUE
        ,INSERT_DATE
    )
    VALUES (
        @ELR_MESSAGE_OBX_ID
        ,~/message/PATIENT_RESULT_GROUP/ORDER_OBSERVATION_GROUP/OBSERVATION_GROUP/OBX5_GROUP/rowELR_MESSAGE_OBX5/OBX_5_COMPONENT_NUMBER~
        ,~/message/PATIENT_RESULT_GROUP/ORDER_OBSERVATION_GROUP/OBSERVATION_GROUP/OBX5_GROUP/rowELR_MESSAGE_OBX5/RELATED_VALUE~
        ,SYSDATETIME()
    )</sql>
    </statement>
    </statement>
    </statement>
    </statement>
    </statement>
</config>

```

Destinations

Name	Page
 js Restore Original Msg..... JavaScript (v.1)	214

**no-op Route Error Handler In**

Filter

No-operation

On ECLIR - ELR 2.3.1 Message Parsing route

Notes

Purpose: This No-operation filter receives messages from the *ECLIR - ELR Parsing Route Error Handler* when an unhandled error occurs anywhere in the route.

Destinations

Name	Page
 js Set Error properties..... JavaScript (v.1)	198

**no-op ELR Msgs In**

Filter

No-operation

On ECLIR - ELR 2.3.1 Message Parsing route

Notes

Purpose: This No-operation filter receives HL7 v2.3.1 ELR messages from the *ECLIR - ELR Message Parsing* route.

Destinations

Name	Page
js Get JavaScript counter for Group Order ID.....JavaScript (v.1)	212

**Error Queue**

Filter

Error Queue

On ECLIR - ELR 2.3.1 Message Parsing route

Configuration Properties

Error Message	Sent from error filter
---------------	------------------------

**js Get JavaScript counter for Group Order ID**

Filter

JavaScript (v.1)

On ECLIR - ELR 2.3.1 Message Parsing route

Notes

Purpose: In this JavaScript filter:

The JavaScript counter 'cnt_Order_Group_ID' is read from the Rhapsody engine and saved into a variable. This is a counter that is maintained in the Rhapsody engine and is independent of any route activity unless a call is made to increment the counter.

The value of the variable is incremented by 1.

The variable is assigned to the message property newOrderGroupId that is used in the Database Lookup filter *db Create Order Group IDs* in case it is needed for a new order group ID.

Javascript

```
//
=====
//_____ INITIAL VARIABLES (start
)
//
=====
```

```
var next = output.append(input[0]);
```

Javascript

```
//
=====
// _____ INITIAL VARIABLES (end
)
//
=====

//
=====
// _____ GET JavaScript COUNTER FOR Group Order ID (start
)
//
=====

var cnt_Order_Group_ID = getCounter('cnt_Order_Group_ID');
cnt_Order_Group_ID++;
next.setProperty('newOrderGroupId', cnt_Order_Group_ID);

var msgBody = next.text;
msgBody = msgBody.replace(/\x0a/g, "");
next.text = msgBody;

//
=====
// _____ GET JavaScript COUNTER FOR Group Order ID (end
)
//
=====
```

Destinations

Name	Page
 db Query Order Group IDs..... Database LookUp	197

**no-op REH to Sink**

Filter

No-operation

On ECLIR - ELR 2.3.1 Message Parsing route

Notes

Purpose: This No-operation filter sends messages from the *ECLIR - ELR Parsing Route Error Handler* to the *ECLIR - ELR Route Error Handler Sink* after the errors have been stored into the ELR_ERROR table in the ECLIR database.

Destinations

Name	Page
 ECLIR - ELR Route Error Handler Sink..... Sink	13



js Restore Original Msg

Filter

JavaScript (v.1)

On ECLIR - ELR 2.3.1 Message Parsing route

Notes

Purpose: This JavaScript filter retrieves the original ELR message from the snapshot ID captured in the *ss Original Copy of ELR message Snapshot* filter and restores it to the route for further processing. The message property for this ID is SnapshotIDofOrigMsg. It also clears the message property ssOriginalMsg to reduce memory use in the Rhapsody datastore.

Javascript

```
//
=====
// _____ INITIAL VARIABLES (start
)
//
=====

var next = output.append(input[0]);

//
=====
// _____ INITIAL VARIABLES (end
)
//
=====

//
=====
// _____ RELOAD ORIGINAL MESSAGE FOR VALIDATION ROUTE (start
)
//
=====
// Reload the original message from the Snapshot and clear the message property containing the
message.

if (next.getProperty("SnapshotIDofOrigMsg") != null) {
    next.text = next.loadMessageSnapshotForId(next.getProperty("SnapshotIDofOrigMsg")).text;
    next.setProperty('ssOriginalMsg', '');

    var msgBody = next.text;
    msgBody = msgBody.replace(/\x0a/g, "");
    next.text = msgBody;
}
```

Javascript

```

}

//
=====
// _____ RELOAD ORIGINAL MESSAGE FOR VALIDATION ROUTE (end
)
//
=====

```

Destinations

Name	Page
 no-op ELR Out to Validation..... No-operation	232



db ELR Parsing Route Errors

Filter

Database LookUp

On ECLIR - ELR 2.3.1 Message Parsing route

Notes

Purpose: This Database Lookup filter inserts an error record into the ELR_ERROR table in the ECLIR database in the event an unhandled error is caught by the *ECLIR - ELR Parsing Route Error Handler*.

Configuration Properties

Maximum Concurrency	1	
Database	[REDACTED]	
Host	[REDACTED]	
Port	[REDACTED]	
Database Name	[REDACTED]	
Username	[REDACTED]	
Password	[REDACTED]	
Additional Connection Properties	Property Name	Property Value
	[REDACTED]	[REDACTED]
Configuration File	[REDACTED]	
Message Type	Plain text	
Definition		
Query Timeout		
Deadlock Retry Attempts	5	
Deadlock Retry Delay	100	
Socket Timeout	0	

Configuration File: Database LookUp 2.xml

```

<statement name="ELR_MESSAGE ID Does Not exist" ide:type="executeSql" test="Null($ELR_MESSAGE_ID)" ide:summary="The ELR_MESSAGE ID does not exist in the ELR_MESSAGE table. Create an entry in that table to create an ID." ide:allowCompoundStatements="true">
  <sql>INSERT INTO ELR_MESSAGE (
    ORIGINAL_MESSAGE
  , [REDACTED] ENVIRONMENT
  , SUCCESS_FL
  , INSERT_DATE
  )
VALUES (
  $messageContent
  , [REDACTED] ENVIRONMENT
  , 0
  , SYSDATETIME()
  )</sql>
  <statement ide:type="compoundSqlStatement">
    <sql>SELECT \@ \@IDENTITY AS ELR_MESSAGE_ID</sql>
    <substituteColumn name="ELR_MESSAGE_ID">$ELR_MESSAGE_ID</substituteColumn>
  </statement>
  <statement name="Insert into ELR_ERROR table" ide:type="executeSql" test="NotNull(@ELR_MESSAGE_ID)" ide:summary="Add error record to the ELR_ERROR table." ide:allowCompoundStatements="true">
    <sql>INSERT INTO ELR_ERROR (
      ELR_MESSAGE_ID
    , VALIDATION_TYPE
    , ERROR_CATEGORY
    , ERROR_TYPE
    , PROCESSING_STEP
    , ERROR_MSG
    , INSERT_DATE
    )
VALUES (
  @ELR_MESSAGE_ID
  , $VALIDATION_TYPE
  , $ERROR_CATEGORY
  , $ERROR_TYPE
  , $PROCESSING_STEP
  , $ERROR_MSG
  , SYSDATETIME()
  )</sql>
  </statement>
</statement>
  <statement name="Insert into ELR_ERROR table" ide:type="executeSql" test="NotNull($ELR_MESSAGE_ID)" ide:summary="Add error record to the ELR_ERROR table." ide:allowCompoundStatements="true">
    <sql>INSERT INTO ELR_ERROR (
      ELR_MESSAGE_ID
    , VALIDATION_TYPE
    , ERROR_CATEGORY
    , ERROR_TYPE
    , PROCESSING_STEP
    , ERROR_MSG
    , INSERT_DATE
    )

```

Configuration File: Database LookUp 2.xml

```
VALUES (
  $ELR_MESSAGE_ID
  , $VALIDATION_TYPE
  , $ERROR_CATEGORY
  , $ERROR_TYPE
  , $PROCESSING_STEP
  , $ERROR_MSG
  , SYSDATETIME()
) </sql>
</statement>
</config>
```

Destinations

Name	Page
 no-op REH to Sink..... No-operation	213

**js Parse ELR to XML for DB Inserts**

Filter

JavaScript (v.1)

On ECLIR - ELR 2.3.1 Message Parsing route

Notes

Purpose: In this JavaScript filter:

The main JavaScript code traverses each segment and ordered group according to the message definition ECLIR_ELOR_ORU_2.3.1_Parsing.s3d for an ORU – Unsolicited Observation Message (Event R01). This message definition follows HL7.org v2.3.1 standards message structure but allows for longer field lengths in order to capture any fields that may exceed the field length limit.

Data is parsed from the message and written to an XML flat file in order to easily iterate through elements to store data to the ELR tables in ECLIR when sent to the *db Insert ELR parsed data into ECLIR Database Lookup* filter.

Javascript

```
//
=====
// _____ INITIAL VARIABLES (start
)
//
=====

var next = output.append(input[0]);
var dbInsertsXML = new XML(<message></message>);
var tempEle;
var today = new Date();
var errorDB_failed_assertion_location = [];
var errorDB_failedAssertID = [];
var errorDB_error_detail = [];
var errorDB_error_count = [];
var errArrayIdx = 0;
var arrFailedAsserts = [];
```

Javascript

```

var arrFailedAssertSchType = [];
var arrFailedAsserts_Len = 0;
var errorFileFormat = false;
var msgDateTime = next.getField('MSH.DateTimeOfMessage');

//
=====
//_____INITIAL VARIABLES (end
)
//
=====

//
=====
//_____FUNCTIONS (start
)
//
=====

//FUNCTION: fnDBColumnInsert
//Function to insert node into output XML.
function fnDBColumnInsert(nameValue, columnValue) {
  if (columnValue != null) {
    var dbColumnNode = <{nameValue}>{columnValue}</{nameValue}>;
    tempEle.appendChild(dbColumnNode);
  }
}

//FUNCTION: fnTruncateToMax
//Function to truncate the strField to the max length intMax.
function fnTruncateToMax(strFieldPath, intMax) {
  if (next.getField(strFieldPath) != null) {
    var strField = next.getField(strFieldPath).substring(0, intMax);
    return strField;
  }
}

//FUNCTION: fnPadZeros
//--- Function to pad a number with leading zero.
function fnPadZeros(num, size) {
  num = num.toString();
  while (num.length < size) num = "0" + num;
  return num;
}

//FUNCTION: fnGetDateTimeComponents
//--- Function to extract individual components from a Date/Time value.
function fnGetDateTimeComponents (inputDateTime) {
  var inputYear = '';
  var inputMonth = '';
  var inputDay = '';
  var inputHours = '';

```

Javascript

```

var inputMinutes = '';
var inputSeconds = '';
var inputOffsetHours = 0;
var inputOffsetMinutes = 0;
var inputOffsetSign = '';
if (inputDateTime != null) {
  if (inputDateTime.length >= 8) {
    if (inputDateTime.match(/^(?:(?:?:?:?:?:[13579][26]|[2468][048])00)|(?:[0-9]{2}(?:?:[13579][26])|(?:[2468][048]|0[48])))(?:(?:?:09|04|06|11)(?:0[1-9]|1[0-9]|2[0-9]|30))|(?:?:01|03|05|07|08|10|12)(?:0[1-9]|1[0-9]|2[0-9]|3[01]))|(?:02(?:0[1-9]|1[0-9]|2[0-9]))|(?:[0-9]{4}(?:?:?:09|04|06|11)(?:0[1-9]|1[0-9]|2[0-9]|30))|(?:?:01|03|05|07|08|10|12)(?:0[1-9]|1[0-9]|2[0-9]|3[01]))|(?:02(?:[01][0-9]|2[0-8])))(?:0[0-9]|1[0-9]|2[0-3])?(?:[0-5][0-9]){2})?(\.\d)?\d?\d?\d?((\+|-)[0-1][0-9]{3})?$/)) != null) {
      inputYear = fnPadZeros(inputDateTime.slice(0, 4), 4);
      inputMonth = fnPadZeros(inputDateTime.slice(4, 6), 2);
      inputDay = fnPadZeros(inputDateTime.slice(6, 8), 2);
      inputHours = fnPadZeros(inputDateTime.slice(8, 10), 2);
      inputMinutes = fnPadZeros(inputDateTime.slice(10, 12), 2);
      inputSeconds = fnPadZeros(inputDateTime.slice(12, 14), 2);
      indexOffsetSign = inputDateTime.lastIndexOf('+');
      if (indexOffsetSign == -1) { indexOffsetSign = inputDateTime.lastIndexOf('-'); }

      if (indexOffsetSign != -1) { //formats with time-zone offsets
        inputOffsetSign = inputDateTime.substr(indexOffsetSign, 1);
        indexOffsetSign++;
        inputOffsetHours = inputDateTime.substr(indexOffsetSign, 2);
        indexOffsetSign = indexOffsetSign+2;
        inputOffsetMinutes = inputDateTime.substr(indexOffsetSign, 2);
      }
    }
  }
}
return {
  strYear: inputYear,
    strMonth: inputMonth,
  strDay: inputDay,
  strHours: inputHours,
    strMinutes: inputMinutes,
  strSeconds: inputSeconds,
  offsetSign: inputOffsetSign,
    offsetHours: inputOffsetHours,
    offsetMinutes: inputOffsetMinutes
}
}

```

```
//FUNCTION: fnConvertToDocumentTimeZone
//Function to convert Date/Time fields to the same time zone as the document time.
function fnConvertToDocumentTimeZone(msgDateTime, inYear, inMonth, inDay, inHours, inMinutes,
  inSeconds, inOffsetSign, inOffsetHours, inOffsetMinutes) {
  var inputOffsetMilliseconds = (Number(inOffsetHours) * 60 + Number(inOffsetMinutes)) * 60000;
  if (inOffsetSign == '-') { inputOffsetMilliseconds = inputOffsetMilliseconds * -1; }

  if (next.getField('MSH.DateTimeOfMessage') != null) {
    var documentDateTimeInfo = fnGetDateTimeComponents (msgDateTime);
    var documentOffsetMilliseconds = (Number(documentDateTimeInfo.offsetHours) * 60 + Number(doc
umentDateTimeInfo.offsetMinutes)) * 60000;
```

Javascript

```

    if (documentDateTimeInfo.offsetSign == '-') { documentOffsetMilliseconds = documentOffset
Milliseconds * -1; }

    // Calculate the difference in offsets
    var offsetDifference = documentOffsetMilliseconds - inputOffsetMilliseconds;
    // Convert the date to the reference time zone
    var convertedDate = new Date(new Date(inYear, inMonth, inDay, inHours, inMinutes, inSeconds
).getTime() + offsetDifference);
    }
    return convertedDate;
}

//FUNCTION: fnConvertDateTimeField
//Function to convert Date/Time fields to a format compatible with the database (YYYY-MM-DD HH
:MI:SS).
function fnConvertDateTimeField(fieldDateTime, msgDateTime) {
    var formattedFieldDateTime = fieldDateTime;
    if (fieldDateTime != null) {
        var fieldComponents = fnGetDateTimeComponents (fieldDateTime);
        if (fieldComponents.offsetSign == '-' || fieldComponents.offsetSign == '+') {
            if (msgDateTime != null) {
                var strAdjustedFieldDateTime = new Date(fnConvertToDocumentTimeZone(
                    msgDateTime,
                    fieldComponents.strYear,
                    fieldComponents.strMonth-1,
                    fieldComponents.strDay,
                    fieldComponents.strHours,
                    fieldComponents.strMinutes,
                    fieldComponents.strSeconds,
                    fieldComponents.offsetSign,
                    fieldComponents.offsetHours,
                    fieldComponents.offsetMinutes));
            }
            else {
                var strAdjustedFieldDateTime = new Date(fieldComponents.strYear,
                    fieldComponents.strMonth-1,
                    fieldComponents.strDay,
                    fieldComponents.strHours,
                    fieldComponents.strMinutes,
                    fieldComponents.strSeconds);
            }
            var fieldYear = strAdjustedFieldDateTime.getFullYear();
            var fieldMonth = fnPadZeros(strAdjustedFieldDateTime.getMonth() + 1, 2);
            var fieldDay = fnPadZeros(strAdjustedFieldDateTime.getDate(), 2);
            var fieldHours = fnPadZeros(strAdjustedFieldDateTime.getHours(), 2);
            var fieldMinutes = fnPadZeros(strAdjustedFieldDateTime.getMinutes(), 2);
            var fieldSeconds = fnPadZeros(strAdjustedFieldDateTime.getSeconds(), 2);
        }
        else {
            fieldDateTime = new Date(fieldComponents.strYear, fieldComponents.strMonth-1, fieldComp
onents.strDay, fieldComponents.strHours, fieldComponents.strMinutes, fieldComponents.strSeconds
);
            var fieldYear = fieldDateTime.getFullYear();
            var fieldMonth = fnPadZeros(fieldDateTime.getMonth() + 1, 2);
            var fieldDay = fnPadZeros(fieldDateTime.getDate(), 2);
            var fieldHours = fnPadZeros(fieldDateTime.getHours(), 2);
            var fieldMinutes = fnPadZeros(fieldDateTime.getMinutes(), 2);

```

Javascript

```

    var fieldSeconds = fnPadZeros(fieldDateTime.getSeconds(), 2);
}

// Format the date string
formattedFieldDateTime = fieldYear+'-'+fieldMonth+'-'+fieldDay+' '+fieldHours+':'+field
Minutes+':'+fieldSeconds;
}
return formattedFieldDateTime;
}

//FUNCTION: fnGetOrderGroupID
//Function to get the Order Group ID from a list of value pairs in the property list 'ORDER
_GROUP_IDS'.
function fnGetOrderGroupID(inputFillerOrderNumber) {
    var arrOrderGroupIDs = next.getPropertyAsList('ORDER_GROUP_IDS');
    var iterator = arrOrderGroupIDs.iterator();

    while (iterator.hasNext()) {
        var arrSplitPair = iterator.next().split('==');
        if (arrSplitPair[0] == inputFillerOrderNumber) {
            //If a new Order Group ID was added to the ELR_ORDER_GROUP table, increment the JavaScript
            counter.
            if (getCounter('cnt_Order_Group_ID') < arrSplitPair[1]) { incCounter('cnt_Order_Group_ID',
1); }
            return arrSplitPair[1];
        }
    }
}

//FUNCTION: fnGetList
//Function to get a list of values from a repeating field.
function fnGetList (pathHL7) {
    var arrList = next.getFieldAsList(pathHL7);
    if (arrList != null && arrList != '') {
        var strList = '';

        for (var i = 0; i < arrList.length; i++) {
            if (strList != '') { strList += '; '; }
            strList += arrList[i];
        }

        return strList;
    }
}

//
=====
// _____ FUNCTIONS (end
)
//
=====

```

Javascript

```

//
=====
//_____ ELR_MESSAGE (start
)
//
=====
//Parse data to XML output to write to the ELR_MESSAGE table.

tempEle = new XML(<rowELR_MESSAGE></rowELR_MESSAGE>);

if (next.getProperty('rd_type') == 'SPR') {
    fnDBColumnInsert('ORIGINAL_MESSAGE_FORMAT', 'Spreadsheet');
}
else {
    fnDBColumnInsert('ORIGINAL_MESSAGE_FORMAT', fnTruncateToMax('MSH.VersionID.VersionID',50));
}

fnDBColumnInsert('ENVIRONMENT', 'STAGING');
fnDBColumnInsert('SUCCESS_FL', 1);
fnDBColumnInsert('MSH_1', fnTruncateToMax('MSH.FieldSeparator', 10));
fnDBColumnInsert('MSH_2', fnTruncateToMax('MSH.EncodingCharacters',10));
fnDBColumnInsert('MSH_3_1', fnTruncateToMax('MSH.SendingApplication.NamespaceID',50));
fnDBColumnInsert('MSH_3_2', fnTruncateToMax('MSH.SendingApplication.UniversalID',500));
fnDBColumnInsert('MSH_3_3', fnTruncateToMax('MSH.SendingApplication.UniversalIDType',10));
fnDBColumnInsert('MSH_4_1', fnTruncateToMax('MSH.SendingFacility.NamespaceID',50));
fnDBColumnInsert('MSH_4_2', fnTruncateToMax('MSH.SendingFacility.UniversalID',500));
fnDBColumnInsert('MSH_4_3', fnTruncateToMax('MSH.SendingFacility.UniversalIDType',10));
fnDBColumnInsert('MSH_5_1', fnTruncateToMax('MSH.ReceivingApplication.NamespaceID',50));
fnDBColumnInsert('MSH_5_2', fnTruncateToMax('MSH.ReceivingApplication.UniversalID',500));
fnDBColumnInsert('MSH_5_3', fnTruncateToMax('MSH.ReceivingApplication.UniversalIDType',10));
fnDBColumnInsert('MSH_6_1', fnTruncateToMax('MSH.ReceivingFacility.NamespaceID',50));
fnDBColumnInsert('MSH_6_2', fnTruncateToMax('MSH.ReceivingFacility.UniversalID',500));
fnDBColumnInsert('MSH_6_3', fnTruncateToMax('MSH.ReceivingFacility.UniversalIDType',10));
fnDBColumnInsert('MSH_7', fnTruncateToMax('MSH.DateTimeOfMessage',50));
fnDBColumnInsert('MSH_7 CONVERTED', fnConvertDateTimeField(msgDateTime));
fnDBColumnInsert('MSH_9_1', fnTruncateToMax('MSH.MessageType.MessageCode',10));
fnDBColumnInsert('MSH_9_2', fnTruncateToMax('MSH.MessageType.TriggerEvent',10));
fnDBColumnInsert('MSH_9_3', fnTruncateToMax('MSH.MessageType.MessageStructure',10));
fnDBColumnInsert('MSH_10', fnTruncateToMax('MSH.MessageControlID',250));
fnDBColumnInsert('MSH_11', fnTruncateToMax('MSH.ProcessingID',10));
fnDBColumnInsert('MSH_12_1', fnTruncateToMax('MSH.VersionID.VersionID',75));
fnDBColumnInsert('MSH_15', fnTruncateToMax('MSH.AcceptAcknowledgmentType',10));
fnDBColumnInsert('MSH_16', fnTruncateToMax('MSH.ApplicationAcknowledgmentType',10));

dbInsertsXML.appendChild(tempEle);

//
=====
//_____ ELR_MESSAGE (end
)
//
=====

//
=====

```

Javascript

```

// _____ PATIENT_RESULT GROUP (start
)
//
=====
//Parse data to XML output to write to the ELR_MESSAGE_PID table.

for (var i = 0; i < next.getRepeatCount('PATIENT_RESULT'); i++) {
    var tempPATIENT_RESULT_GROUP = new XML(<PATIENT_RESULT_GROUP></PATIENT_RESULT_GROUP>);
    var tempPATIENT_GROUP = new XML(<PATIENT></PATIENT>);
    tempEle = new XML(<rowELR_MESSAGE_PID></rowELR_MESSAGE_PID>);

    fnDBColumnInsert('PID_1', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.PID.SetID',10));
    fnDBColumnInsert('PID_3', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.PID.PatientIdentifier
List',500));
    fnDBColumnInsert('PID_3_1_RE1', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.PID.PatientIden
tifierList[0]/IDNumber',150));
    fnDBColumnInsert('PID_3_5_RE1', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.PID.PatientIden
tifierList[0]/IdentifierTypeCode',50));
    fnDBColumnInsert('PID_3_1_RE2', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.PID.PatientIden
tifierList[1]/IDNumber',150));
    fnDBColumnInsert('PID_3_5_RE2', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.PID.PatientIden
tifierList[1]/IdentifierTypeCode',50));
    fnDBColumnInsert('PID_3_1_RE3', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.PID.PatientIden
tifierList[2]/IDNumber',150));
    fnDBColumnInsert('PID_3_5_RE3', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.PID.PatientIden
tifierList[2]/IdentifierTypeCode',50));
    fnDBColumnInsert('PID_5', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.PID.PatientName
',500));
    fnDBColumnInsert('PID_6_1_1', fnGetList('PATIENT_RESULT['+i+'].PATIENT.PID.MothersMaidenName
[*].FamilyName'));
    fnDBColumnInsert('PID_7', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.PID.DateTimeOfBirth
',50));
    fnDBColumnInsert('PID_7_CONVERTED', fnConvertDateTimeField(next.getField('PATIENT_RESULT['+i
+'].PATIENT.PID.DateTimeOfBirth')));
    fnDBColumnInsert('PID_8', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.PID.AdministrativeSex
',25));
    fnDBColumnInsert('PID_10', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.PID.Race',2500));
    fnDBColumnInsert('PID_10_1_LIST', fnGetList('PATIENT_RESULT['+i+'].PATIENT.PID.Race[*].Ide
ntifier'));
    fnDBColumnInsert('PID_10_2_LIST', fnGetList('PATIENT_RESULT['+i+'].PATIENT.PID.Race[*].Text
'));
    fnDBColumnInsert('PID_10_3_LIST', fnGetList('PATIENT_RESULT['+i+'].PATIENT.PID.Race[*].NameOfC
odingSystem'));
    if (next.getRepeatCount('PATIENT_RESULT['+i+'].PATIENT.PID.Race') > 1) {
        fnDBColumnInsert('PID_10_MULTI_FL', 1);
    }
    else { fnDBColumnInsert('PID_10_MULTI_FL', 0); }
    fnDBColumnInsert('PID_11', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.PID.PatientAddress
',1500));
    fnDBColumnInsert('PID_11_PRIMARY_STATE', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.PID
.PatientAddress[0].StateOrProvince',100));
    fnDBColumnInsert('PID_11_PRIMARY_COUNTY', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.PID
.PatientAddress[0].CountyParishCode',50));

    fnDBColumnInsert('PID_13', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.PID.HomePhoneNumber
',500));
    fnDBColumnInsert('PID_14', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.PID.BusinessPhone
Number',500));

```

Javascript

```

fnDBColumnInsert('PID_15', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.PID.PrimaryLanguage',500));
fnDBColumnInsert('PID_16', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.PID.MaritalStatus',500));
fnDBColumnInsert('PID_17', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.PID.Religion',500));
fnDBColumnInsert('PID_18_1', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.PID.PatientAccountNumber.IDNumber',500));
fnDBColumnInsert('PID_21_1_LIST', fnGetList('PATIENT_RESULT['+i+'].PATIENT.PID.MothersIdentifier[*].IDNumber'));
fnDBColumnInsert('PID_22', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.PID.EthnicGroup',2500));
fnDBColumnInsert('PID_22_1_LIST', fnGetList('PATIENT_RESULT['+i+'].PATIENT.PID.EthnicGroup[*].Identifier'));
fnDBColumnInsert('PID_22_2_LIST', fnGetList('PATIENT_RESULT['+i+'].PATIENT.PID.EthnicGroup[*].Text'));
fnDBColumnInsert('PID_22_3_LIST', fnGetList('PATIENT_RESULT['+i+'].PATIENT.PID.EthnicGroup[*].NameOfCodingSystem'));
if (next.getRepeatCount('PATIENT_RESULT['+i+'].PATIENT.PID.EthnicGroup') > 1) {
  fnDBColumnInsert('PID_22_MULTI_FL', 1);
}
else { fnDBColumnInsert('PID_22_MULTI_FL', 0); }
fnDBColumnInsert('PID_23', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.PID.BirthPlace',500));
fnDBColumnInsert('PID_24', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.PID.MultipleBirthIndicator',10));
fnDBColumnInsert('PID_25', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.PID.BirthOrder',10));
fnDBColumnInsert('PID_26', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.PID.Citizenship',500));
fnDBColumnInsert('PID_27', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.PID.VeteransMilitaryStatus',500));
fnDBColumnInsert('PID_29', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.PID.PatientDeathDateAndTime',50));
fnDBColumnInsert('PID_29_CONVERTED', fnConvertDateTimeField(next.getField('PATIENT_RESULT['+i+'].PATIENT.PID.PatientDeathDateAndTime'), msgDateTime));
fnDBColumnInsert('PID_30', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.PID.PatientDeathIndicator',10));
if (next.getRepeatCount('PATIENT_RESULT['+i+'].PATIENT.NTE') >= 1) { fnDBColumnInsert('NTE_INCLUDED_FL', 1); }
else { fnDBColumnInsert('NTE_INCLUDED_FL', 0); }
fnDBColumnInsert('PV1_1', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.VISIT.PV1.SetID',50));
fnDBColumnInsert('PV1_2', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.VISIT.PV1.PatientClass',20));
fnDBColumnInsert('PV1_4', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.VISIT.PV1.AdmissionType',20));
fnDBColumnInsert('PV1_14', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.VISIT.PV1.AdmitSource',20));
fnDBColumnInsert('PV1_44', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.VISIT.PV1.AdmitDateTime',50));
fnDBColumnInsert('PV1_44_CONVERTED', fnConvertDateTimeField(next.getField('PATIENT_RESULT['+i+'].PATIENT.VISIT.PV1.AdmitDateTime'), msgDateTime));
fnDBColumnInsert('PV1_45', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.VISIT.PV1.DischargeDateTime[0]',50));
fnDBColumnInsert('PV1_45_CONVERTED', fnConvertDateTimeField(next.getField('PATIENT_RESULT['+i+'].PATIENT.VISIT.PV1.DischargeDateTime[0]'), msgDateTime));

```

Javascript

```

tempPATIENT_GROUP.appendChild(tempEle);

//
=====
//_____ NK1 GROUP (start
)
//
=====
//Parse data to XML output to write to the ELR_MESSAGE_NK1 table.

var tempNK1_GROUP = new XML(<NK1_GROUP></NK1_GROUP>);
for (var j = 0; j < next.getRepeatCount('PATIENT_RESULT['+i+'].PATIENT.NK1'); j++) {
    tempEle = new XML(<rowELR_MESSAGE_NK1></rowELR_MESSAGE_NK1>);

    fnDBColumnInsert('NK1_1', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.NK1['+j+'].SetID
',10));
    fnDBColumnInsert('NK1_2', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.NK1['+j+'].Name
',2500));
    fnDBColumnInsert('NK1_3_1', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.NK1['+j+'].Relatio
nship.Identifier',50));
    fnDBColumnInsert('NK1_3_2', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.NK1['+j+'].Relatio
nship.Text',500));
    fnDBColumnInsert('NK1_3_3', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.NK1['+j+'].Relatio
nship.NameOfCodingSystem',25));
    fnDBColumnInsert('NK1_4', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.NK1['+j+'].Address
',1500));
    fnDBColumnInsert('NK1_5', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.NK1['+j+'].Phone
Number',500));
    fnDBColumnInsert('NK1_13', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.NK1['+j+'].Organiz
ationName',500));
    fnDBColumnInsert('NK1_20', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.NK1['+j+'].Primary
Language',500));
    fnDBColumnInsert('NK1_30', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.NK1['+j+'].Contact
PersonsName',500));
    fnDBColumnInsert('NK1_31', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.NK1['+j+'].Contact
PersonsTelephoneNumber',500));
    fnDBColumnInsert('NK1_32', fnTruncateToMax('PATIENT_RESULT['+i+'].PATIENT.NK1['+j+'].Contact
PersonsAddress',1500));

    tempNK1_GROUP.appendChild(tempEle);
}
tempPATIENT_GROUP.appendChild(tempNK1_GROUP);

//
=====
//_____ NK1 GROUP (end
)
//
=====

tempPATIENT_RESULT_GROUP.appendChild(tempPATIENT_GROUP);

//
=====

```

Javascript

```
// _____ ORDER_OBSERVATION GROUP (start
)
//
=====
//Parse data to XML output to write to the ELR_MESSAGE_OBR table.

for (var k = 0; k < next.getRepeatCount('PATIENT_RESULT['+i+'].ORDER_OBSERVATION'); k++) {
  var tempORDER_OBSERVATION_GROUP = new XML(<ORDER_OBSERVATION_GROUP></ORDER_OBSERVATION_GROUP>);
  tempEle = new XML(<rowELR_MESSAGE_OBR></rowELR_MESSAGE_OBR>);
  var ORDER_GROUP_ID = '';
  ORDER_GROUP_ID = fnGetOrderGroupID(next.getField('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].OBR.FillerOrderNumber.EntityIdentifier'));
  fnDBColumnInsert('FILLER_ORDER_NUMBER', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].OBR.FillerOrderNumber.EntityIdentifier', 500));
  fnDBColumnInsert('ORDER_GROUP_ID', ORDER_GROUP_ID);
  fnDBColumnInsert('ORC_1', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].ORC.OrderControl', 10));
  fnDBColumnInsert('ORC_2_1', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].ORC.PlacerOrderNumber.EntityIdentifier', 250));
  fnDBColumnInsert('ORC_2_2', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].ORC.PlacerOrderNumber.NamespaceID', 25));
  fnDBColumnInsert('ORC_2_3', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].ORC.PlacerOrderNumber.UniversalID', 250));
  fnDBColumnInsert('ORC_2_4', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].ORC.PlacerOrderNumber.UniversalIDType', 15));
  fnDBColumnInsert('ORC_3_1', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].ORC.FillerOrderNumber.EntityIdentifier', 250));
  fnDBColumnInsert('ORC_3_2', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].ORC.FillerOrderNumber.NamespaceID', 25));
  fnDBColumnInsert('ORC_3_3', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].ORC.FillerOrderNumber.UniversalID', 250));
  fnDBColumnInsert('ORC_3_4', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].ORC.FillerOrderNumber.UniversalIDType', 15));
  fnDBColumnInsert('ORC_4_1', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].ORC.PlacerGroupNumber.EntityIdentifier', 250));
  fnDBColumnInsert('ORC_5', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].ORC.OrderStatus', 10));
  fnDBColumnInsert('ORC_6', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].ORC.ResponseFlag', 10));
  fnDBColumnInsert('ORC_8', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].ORC.Parent', 500));
  fnDBColumnInsert('ORC_9', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].ORC.DateTimeOfTransaction', 50));
  fnDBColumnInsert('ORC_10', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].ORC.EnteredBy', 500));
  fnDBColumnInsert('ORC_11', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].ORC.VerifiedBy', 500));
  fnDBColumnInsert('ORC_12', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].ORC.OrderingProvider', 500));
  fnDBColumnInsert('ORC_13', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].ORC.EnterersLocation', 150));
  fnDBColumnInsert('ORC_14', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].ORC.CallBackPhoneNumber', 1500));
  fnDBColumnInsert('ORC_15', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].ORC.OrderEffectiveDateTime', 50));
  fnDBColumnInsert('ORC_16', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].ORC.OrderControlCodeReason', 500));
}
```

Javascript

```

    fnDBColumnInsert('ORC_17', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].ORC.EnteringOrganization', 500));
    fnDBColumnInsert('ORC_18', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].ORC.EnteringDevice', 500));
    fnDBColumnInsert('ORC_19', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].ORC.ActionBy', 500));
    fnDBColumnInsert('ORC_21', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].ORC.OrderingFacilityName', 1500));
    fnDBColumnInsert('ORC_22', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].ORC.OrderingFacilityAddress', 1500));
    fnDBColumnInsert('ORC_23', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].ORC.OrderingFacilityPhoneNumber', 1500));
    fnDBColumnInsert('ORC_24', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].ORC.OrderingProviderAddress', 1500));

    fnDBColumnInsert('OBR_1', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].OBR
.SetID', 10));
    fnDBColumnInsert('OBR_2_1', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.PlacerOrderNumber.EntityIdentifier', 250));
    fnDBColumnInsert('OBR_2_2', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.PlacerOrderNumber.NamespaceID', 25));
    fnDBColumnInsert('OBR_2_3', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.PlacerOrderNumber.UniversalID', 250));
    fnDBColumnInsert('OBR_2_4', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.PlacerOrderNumber.UniversalIDType', 15));
    fnDBColumnInsert('OBR_3_1', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.FillerOrderNumber.EntityIdentifier', 250));
    fnDBColumnInsert('OBR_3_2', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.FillerOrderNumber.NamespaceID', 25));
    fnDBColumnInsert('OBR_3_3', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.FillerOrderNumber.UniversalID', 250));
    fnDBColumnInsert('OBR_3_4', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.FillerOrderNumber.UniversalIDType', 15));
    fnDBColumnInsert('OBR_4_1', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.UniversalServiceIdentifier.Identifier', 50));
    fnDBColumnInsert('OBR_4_2', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.UniversalServiceIdentifier.Text', 250));
    fnDBColumnInsert('OBR_4_3', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.UniversalServiceIdentifier.NameOfCodingSystem', 25));
    fnDBColumnInsert('OBR_4_4', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.UniversalServiceIdentifier.AlternateIdentifier', 50));
    fnDBColumnInsert('OBR_4_5', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.UniversalServiceIdentifier.AlternateText', 250));
    fnDBColumnInsert('OBR_4_6', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.UniversalServiceIdentifier.NameOfAlternateCodingSystem', 25));
    fnDBColumnInsert('OBR_7', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].OBR
.ObservationDateTime', 50));
    fnDBColumnInsert('OBR_7_CONVERTED', fnConvertDateTimeField(next.getField('PATIENT_RESULT['+i
+'].ORDER_OBSERVATION['+k+'].OBR.ObservationDateTime'), msgDateTime));
    fnDBColumnInsert('OBR_8', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].OBR
.ObservationEndDateTime', 50));
    fnDBColumnInsert('OBR_10', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.CollectorIdentifier', 500));
    fnDBColumnInsert('OBR_11', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.SpecimenActionCode', 50));
    fnDBColumnInsert('OBR_12_1', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.DangerCode.Identifier', 10));

```

Javascript

```

fnDBColumnInsert('OBR_12_2', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.DangerCode.Text', 250));
fnDBColumnInsert('OBR_13', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.RelevantClinicalInformation', 500));
fnDBColumnInsert('OBR_15', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.SpecimenSource', 1500));
fnDBColumnInsert('OBR_15_1_1', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.SpecimenSource.SpecimenSourceNameorCode.Identifier', 150));
fnDBColumnInsert('OBR_15_1_2', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.SpecimenSource.SpecimenSourceNameorCode.Text', 250));
fnDBColumnInsert('OBR_15_1_3', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.SpecimenSource.SpecimenSourceNameorCode.NameOfCodingSystem', 250));
fnDBColumnInsert('OBR_15_4_1', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.SpecimenSource.BodySite.Identifier', 150));
fnDBColumnInsert('OBR_15_4_2', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.SpecimenSource.BodySite.Text', 250));
fnDBColumnInsert('OBR_15_4_3', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.SpecimenSource.BodySite.NameOfCodingSystem', 250));
fnDBColumnInsert('OBR_16', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.OrderingProvider', 500));
fnDBColumnInsert('OBR_17', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.OrderCallbackPhoneNumber', 500));
fnDBColumnInsert('OBR_18', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.PlacerField1', 100));
fnDBColumnInsert('OBR_19', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.PlacerField2', 100));
fnDBColumnInsert('OBR_20', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.FillerField1', 100));
fnDBColumnInsert('OBR_21', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.FillerField2', 100));
fnDBColumnInsert('OBR_22', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.ResultsRptStatusChngDateTime', 50));
fnDBColumnInsert('OBR_22_CONVERTED', fnConvertDateTimeField(next.getField('PATIENT_RESULT['+i
+'].ORDER_OBSERVATION['+k+'].OBR.ResultsRptStatusChngDateTime'), msgDateTime));
fnDBColumnInsert('OBR_24', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.DiagnosticServSectID', 10));

fnDBColumnInsert('OBR_25', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.ResultStatus', 10));
fnDBColumnInsert('OBR_26_1_1', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.ParentResult.ParentObservationIdentifier.Identifier', 50));
fnDBColumnInsert('OBR_26_1_2', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.ParentResult.ParentObservationIdentifier.Text', 500));
fnDBColumnInsert('OBR_26_1_3', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.ParentResult.ParentObservationIdentifier.NameOfCodingSystem', 25));
fnDBColumnInsert('OBR_26_1_4', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.ParentResult.ParentObservationIdentifier.AlternateIdentifier', 50));
fnDBColumnInsert('OBR_26_1_5', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.ParentResult.ParentObservationIdentifier.AlternateText', 500));
fnDBColumnInsert('OBR_26_1_6', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.ParentResult.ParentObservationIdentifier.NameOfAlternateCodingSystem', 25));
fnDBColumnInsert('OBR_26_2', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.ParentResult.ParentObservationSubidentifier', 25));
fnDBColumnInsert('OBR_26_3', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.ParentResult.ParentObservationValueDescriptor', 500));
fnDBColumnInsert('OBR_28', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.ResultCopiesTo', 500));
fnDBColumnInsert('OBR_29_2_1', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.Parent.FillerAssignedIdentifier.EntityIdentifier', 500));

```

Javascript

```

    fnDBColumnInsert('OBR_31', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.ReasonForStudy', 1500));
    fnDBColumnInsert('OBR_32_1_1', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.PrincipalResultInterpreter.Name.IDNumber', 30));
    fnDBColumnInsert('OBR_32_1_2', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.PrincipalResultInterpreter.Name.FamilyName', 100));
    fnDBColumnInsert('OBR_32_1_3', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.PrincipalResultInterpreter.Name.GivenName', 100));
    fnDBColumnInsert('OBR_32_1_4', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.PrincipalResultInterpreter.Name.MiddleInitialOrName', 100));
    fnDBColumnInsert('OBR_32_1_5', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.PrincipalResultInterpreter.Name.Suffix', 50));
    fnDBColumnInsert('OBR_32_1_6', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.PrincipalResultInterpreter.Name.Prefix', 50));
    fnDBColumnInsert('OBR_32_1_9', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.PrincipalResultInterpreter.Name.AssigningAuthorityNamespaceID', 50));
    fnDBColumnInsert('OBR_32_1_10', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.PrincipalResultInterpreter.Name.AssigningAuthorityUniversalID', 500));
    fnDBColumnInsert('OBR_32_1_11', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.PrincipalResultInterpreter.Name.AssigningAuthorityUniversalIDType', 10));
    fnDBColumnInsert('OBR_33', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.AssistantResultInterpreter', 500));
    fnDBColumnInsert('OBR_34', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.Technician', 500));
    fnDBColumnInsert('OBR_35', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.Transcriptionist', 500));
    fnDBColumnInsert('OBR_36', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.ScheduledDateTime', 50));
    fnDBColumnInsert('OBR_39', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.CollectorsComment', 1500));
    fnDBColumnInsert('OBR_44', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.ProcedureCode', 500));
    fnDBColumnInsert('OBR_45', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBR.ProcedureCodeModifier', 500));
    if (next.getRepeatCount('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].NTE') >= 1) { fnDBC
olumnInsert('NTE_INCLUDED_FL', 1); }
    else { fnDBColumnInsert('NTE_INCLUDED_FL', 0); }

    tempORDER_OBSERVATION_GROUP.appendChild(tempEle);

    //
=====
    //_____ OBSERVATION GROUP (start
)
    //
=====
    //Parse data to XML output to write to the ELR_MESSAGE_OBX table.

    for (var p = 0; p < next.getRepeatCount('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].OBS
ERVATION'); p++) {
        var tempOBSERVATION_GROUP = new XML(<OBSERVATION_GROUP></OBSERVATION_GROUP>);
        tempEle = new XML(<rowELR_MESSAGE_OBR_OBX></rowELR_MESSAGE_OBR_OBX>);

        fnDBColumnInsert('PARENT_SEGMENT_TYPE', 'O');
        fnDBColumnInsert('OBX_1', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBSERVATION['+p+'].OBX.SetID', 10));
        fnDBColumnInsert('OBX_2', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBSERVATION['+p+'].OBX.ValueType', 10));

```

Javascript

```

    fnDBCColumnInsert('OBX_3_1', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBSERVATION['+p+'].OBX.ObservationIdentifier.Identifier', 50));
    fnDBCColumnInsert('OBX_3_2', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBSERVATION['+p+'].OBX.ObservationIdentifier.Text', 50));
    fnDBCColumnInsert('OBX_3_3', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBSERVATION['+p+'].OBX.ObservationIdentifier.NameOfCodingSystem', 25));
    fnDBCColumnInsert('OBX_3_4', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBSERVATION['+p+'].OBX.ObservationIdentifier.AlternateIdentifier', 50));
    fnDBCColumnInsert('OBX_3_5', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBSERVATION['+p+'].OBX.ObservationIdentifier.AlternateText', 25));
    fnDBCColumnInsert('OBX_3_6', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBSERVATION['+p+'].OBX.ObservationIdentifier.NameOfAlternateCodingSystem', 50));
    fnDBCColumnInsert('OBX_4', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBSERVATION['+p+'].OBX.ObservationSubID', 50));
    fnDBCColumnInsert('OBX_5', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBSERVATION['+p+'].OBX.ObservationValue'));
    fnDBCColumnInsert('OBX_6_1', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBSERVATION['+p+'].OBX.Units.Identifier', 50));
    fnDBCColumnInsert('OBX_6_2', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBSERVATION['+p+'].OBX.Units.Text', 50));
    fnDBCColumnInsert('OBX_6_3', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBSERVATION['+p+'].OBX.Units.NameOfCodingSystem', 25));
    fnDBCColumnInsert('OBX_7', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBSERVATION['+p+'].OBX.ReferencesRange', 150));
    fnDBCColumnInsert('OBX_8', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBSERVATION['+p+'].OBX.AbnormalFlags[0]', 25));
    fnDBCColumnInsert('OBX_9', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBSERVATION['+p+'].OBX.Probability', 25));
    fnDBCColumnInsert('OBX_10', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBSERVATION['+p+'].OBX.NatureOfAbnormalTest[0]', 10));
    fnDBCColumnInsert('OBX_11', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBSERVATION['+p+'].OBX.ObservationResultStatus', 10));
    fnDBCColumnInsert('OBX_12', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBSERVATION['+p+'].OBX.EffectiveDateOfReferenceRangeValues', 50));
    fnDBCColumnInsert('OBX_13', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBSERVATION['+p+'].OBX.UserDefinedAccessChecks', 50));
    fnDBCColumnInsert('OBX_14', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBSERVATION['+p+'].OBX.DateTimeOfTheObservation', 50));
    fnDBCColumnInsert('OBX_14_CONVERTED', fnConvertDateTimeField(next.getField('PATIENT_RESULT
['+i+'].ORDER_OBSERVATION['+k+'].OBSERVATION['+p+'].OBX.DateTimeOfTheObservation'), msgDate
Time));
    fnDBCColumnInsert('OBX_15_1', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBSERVATION['+p+'].OBX.ProducersReference.Identifier', 50));
    fnDBCColumnInsert('OBX_15_2', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBSERVATION['+p+'].OBX.ProducersReference.Text', 50));
    fnDBCColumnInsert('OBX_15_3', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBSERVATION['+p+'].OBX.ProducersReference.NameOfCodingSystem', 25));
    fnDBCColumnInsert('OBX_16', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBSERVATION['+p+'].OBX.ResponsibleObserver[0]', 50));
    fnDBCColumnInsert('OBX_17', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].OBSERVATION['+p+'].OBX.ObservationMethod[0]', 50));
    if (next.getRepeatCount('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].OBSERVATION['+p
+'].NTE') >= 1) { fnDBCColumnInsert('NTE_INCLUDED_FL', 1); }
    else { fnDBCColumnInsert('NTE_INCLUDED_FL', 0); }

    tempOBSERVATION_GROUP.appendChild(tempEle);

//OBX-5

```

Javascript

```

//Create a record for each component in OBX-5 value if it is a complex data type.
// If it is a primitive data type, the array of values will only have one item and only one
record is created.
var strOBX5 = next.getField('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].OBSERVATION['+p
+'].OBX.ObservationValue');
if (strOBX5 != null) {
  var tempOBX5_GROUP = new XML(<OBX5_GROUP></OBX5_GROUP>);

  var arrOBX5 = strOBX5.split('^');
  for (var x = 0; x < arrOBX5.length; x++) {
    if ((arrOBX5[x] != null) && (arrOBX5[x] != '')) {
      tempEle = new XML(<rowELR_MESSAGE_OBX5></rowELR_MESSAGE_OBX5>);
      fnDBCColumnInsert('OBX_5_COMPONENT_NUMBER', x + 1);
      fnDBCColumnInsert('RELATED_VALUE', arrOBX5[x]);
      tempOBX5_GROUP.appendChild(tempEle);
    }
  }
}
//OBX-5 (end)

tempOBSERVATION_GROUP.appendChild(tempOBX5_GROUP);

tempORDER_OBSERVATION_GROUP.appendChild(tempOBSERVATION_GROUP);
}
} // END -- for (var p = 0; p < next.getRepeatCount('PATIENT_RESULT['+i+'].ORDER_OBSERVATION
['+k+'].OBSERVATION'); p++)

//
=====
//_____ OBSERVATION GROUP (end
)
//
=====

tempPATIENT_RESULT_GROUP.appendChild(tempORDER_OBSERVATION_GROUP);

} // for (var k = 0; k < next.getRepeatCount('PATIENT_RESULT['+i+'].ORDER_OBSERVATION'); k++)
(end)

//
=====
//_____ ORDER_OBSERVATION GROUP (end
)
//
=====

dbInsertsXML.appendChild(tempPATIENT_RESULT_GROUP);

} // (end) for (var i = 0; i < next.getRepeatCount('PATIENT_RESULT'); i++)

//
=====
//_____ PATIENT_RESULT GROUP (end
)
//
=====

next.xml = dbInsertsXML;

```

Javascript

```
//
=====
//_____ SET PROPERTY WITH ORIGINAL MESSAGE TO SEND TO VALIDATION ROUTES (start
)_____
//
=====

if (next.getProperty("SnapshotIDofOrigMsg") != null) {
    var ssOriginalMsg = next.loadMessageSnapshotForId(next.getProperty("SnapshotIDofOrigMsg
")).text;
    ssOriginalMsg = ssOriginalMsg.replace(/\x0a/g, "");

    next.setProperty('ssOriginalMsg', ssOriginalMsg);
}

//
=====
//_____ SET PROPERTY WITH ORIGINAL MESSAGE TO SEND TO VALIDATION ROUTES (end
)_____
//
=====
```

Destinations

Name	Page
 db Insert ELR parsed data into ECLIR.....Database LookUp	200



no-op ELR Out to Validation

Filter

No-operation

On ECLIR - ELR 2.3.1 Message Parsing route

Notes

Purpose: This No-operation filter sends HL7 v2.3.1 ELR messages to the next route *ECLIR - ELR Message Validation*.

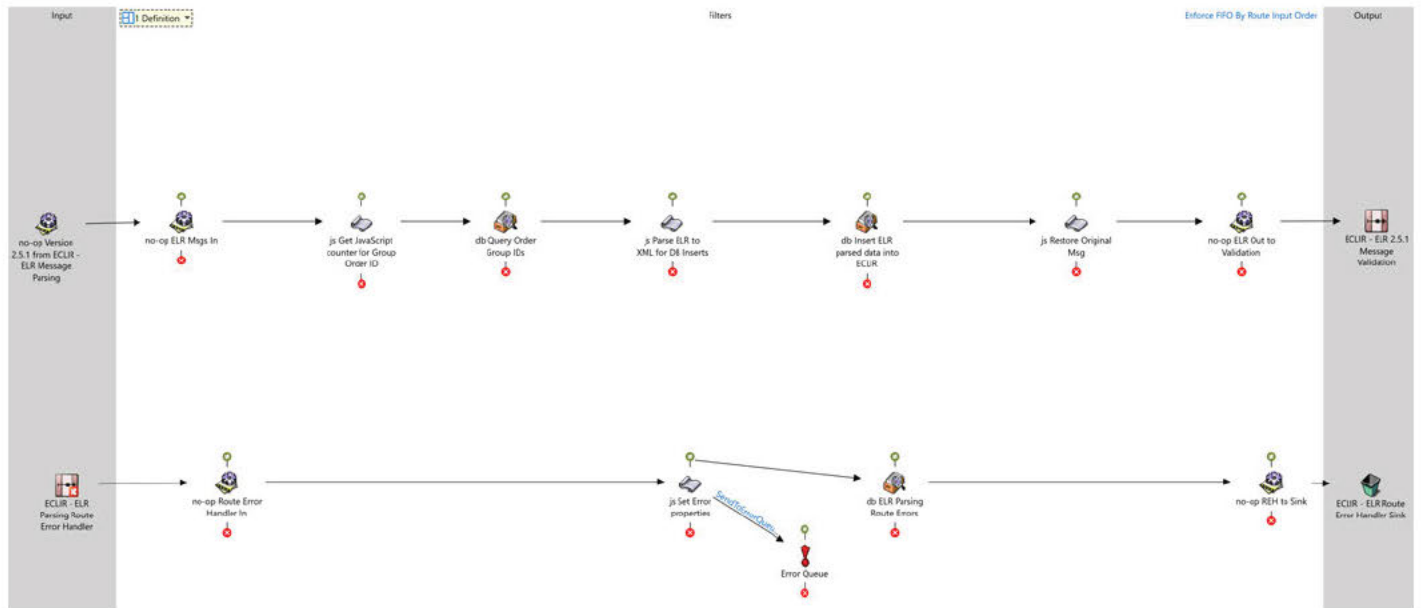
Destinations

Name	Page
 no-op ELR Msgs In.....No-operation	382



ECLIR - ELR 2.5.1 Message Parsing

Route



Notes

Purpose: This route receives HL7 v2.5.1 messages from the *ECLIR - ELR Message Parsing* route. The data in the messages is parsed out and inserted into the relevant ELR tables in the ECLIR database.

Inputs	Page
no-op Version 2.5.1 (filter from "ECLIR - ELR Message Parsing")..... No-operation	294
Outputs	Page
ECLIR - ELR 2.5.1 Message Validation..... Route	407
ECLIR - ELR Route Error Handler Sink..... Sink	13

Filters

Name	Page
db ELR Parsing Route Errors..... Database Lookup	261
db Insert ELR parsed data into ECLIR..... Database Lookup	238
db Query Order Group IDs..... Database Lookup	235
Error Queue..... Error Queue	259
js Get JavaScript counter for Group Order ID..... JavaScript (v.1)	258
js Parse ELR to XML for DB Inserts..... JavaScript (v.1)	263
js Restore Original Msg..... JavaScript (v.1)	259
js Set Error properties..... JavaScript (v.1)	236
no-op ELR Msgs In..... No-operation	257

Name		Page
 no-op ELR Out to Validation.....	No-operation	284
 no-op REH to Sink.....	No-operation	259
 no-op Route Error Handler In.....	No-operation	257

Details

General	
Priority	Normal
Startup State	Startup Third
FIFO Mode	Enforce FIFO By Route Input Order
Stop route on error message	False



db Query Order Group IDs

Filter

Database LookUp

On ECLIR - ELR 2.5.1 Message Parsing route

Notes

Purpose: In this Database Lookup filter:

A SELECT query searches for an existing order group ID from the ELR_ORDER_GROUP table based on the Filler Order Number (OBR-3.1) and the Facility ID (MSH-4.2).

If the ID is found, the following concatenated string is assigned to the Rhapsody message property array ORDER_GROUP_IDS[].

Filler Order Number (OBR-3.1) + "==" + ORDER_GROUP_ID (field)

If the ID is not found, a new one is created using the message property newOrderGroupId as the new ORDER_GROUP_ID field. The new ID is used to create the following concatenated string and is assigned to the Rhapsody message property array ORDER_GROUP_IDS[].

Filler Order Number (OBR-3.1) + "==" + ORDER_GROUP_ID (field)

Configuration Properties

Maximum Concurrency	1	
Database		
Host		
Port		
Database Name		
Username		
Password		
Additional Connection Properties	Property Name	Property Value
Configuration File		
Definition	ECLIR_ELORU_2.5.1_Parsing.s3d	
Query Timeout		
Deadlock Retry Attempts	5	
Deadlock Retry Delay	100	
Socket Timeout	0	
Configuration File:		

```
<statement name="PATIENT_RESULT_GROUP" ide:type="executeSql" iterate="PATIENT_RESULT" ide:summary="Second pass through the message to add/update Order Group IDs." ide:allowCompoundStatements="true" user:iterationRootMessagePath="ORUR01">
```

```
<statement name="Get Order Group ID if it exists" ide:type="executeSql" iterate="PATIENT_RESULT.ORDER_OBSERVATION" ide:summary="Get the Order Group ID based on the MSH-4.2 and the OBR-3.1 if it exists." ide:allowCompoundStatements="true" user:iterationRootMessagePath="ORUR01">
```

```
<sql>SELECT TOP (1) ORDER_GROUP_ID AS OrderGroupID, RELATED_FILLER_ORDER_NUMBER,
CONCAT(RELATED_FILLER_ORDER_NUMBER, '==', ORDER_GROUP_ID) AS ORDER_ID_PAIR
```

Configuration File:

```

FROM ELR_ORDER_GROUP
WHERE RELATED_SENDING_FACILITY = @MSH.SendingFacility.UniversalID
  AND RELATED_FILLER_ORDER_NUMBER = @PATIENT_RESULT.ORDER_OBSERVATION.OBR.FillerOrderNumber
.EntityIdentifier</sql>
  <statement id:type="resultSubstitution" test="NotNull(@ORDER_ID_PAIR)">
    <substituteColumn name="ORDER_ID_PAIR">$ORDER_GROUP_IDS[]</substituteColumn>
  </statement>
  <statement id:type="resultSubstitution" test="NotNull(@OrderGroupId)">
    <substituteColumn name="OrderGroupId">$newOrderGroupId</substituteColumn>
  </statement>
  <resultSet>AllowNullResultSet</resultSet>
  <statement name="Order Group ID not found" id:type="executeSql" test="Null(@ORDER_ID_PAIR)" id:summary="Executes if the Order Group ID was not found." id:allowCompoundStatements="true">
    <statement name="Create ORDER_GROUP_ID" id:type="executeSql" iterate="PATIENT_RESULT.ORDER_OBSERVATION" id:summary="If ORDER_GROUP_ID was not found in the previous query, insert a new record in the ELR_ORDER_GROUP table for the order group." id:allowCompoundStatements="true" user:iterationRootMessagePath="ORUR01">
      <sql>INSERT INTO ELR_ORDER_GROUP (
ORDER_GROUP_ID
,RELATED_FILLER_ORDER_NUMBER
,RELATED_SENDING_FACILITY
)
VALUES (
  $newOrderGroupId
  ,@PATIENT_RESULT.ORDER_OBSERVATION.OBR.FillerOrderNumber.EntityIdentifier
  ,@MSH.SendingFacility.UniversalID
)</sql>
      <statement id:type="compoundSqlStatement">
        <sql>SELECT CONCAT(@PATIENT_RESULT.ORDER_OBSERVATION.OBR.FillerOrderNumber.EntityIdentifier, '==', $newOrderGroupId)
AS ORDER_ID_PAIR_new</sql>
        <statement id:type="resultSubstitution" test="NotNull(@ORDER_ID_PAIR_new)">
          <substituteColumn name="ORDER_ID_PAIR_new">$ORDER_GROUP_IDS[]</substituteColumn>
        </statement>
      </statement>
    </statement>
  </statement>
</statement>
</config>

```

Destinations

Name	Page
js Parse ELR to XML for DB Inserts..... JavaScript (v.1)	263



js Set Error properties

JavaScript (v.1)

On ECLIR - ELR 2.5.1 Message Parsing route

Filter

Notes

Notes

Purpose: This JavaScript filter

Sets the property `VALIDATION_TYPE` with the type of validation error to be stored in the `VALIDATION_TYPE` column in the `ELR_ERROR` table. The type is set to 'Rhapsody' because the error is an internal Rhapsody error.

Sets the property `ERROR_CATEGORY` with the category of the error to be stored in the `ERROR_CATEGORY` column in the `ELR_ERROR` table. The category is set to 'FATAL' because the message stops processing.

Sets the property `ERROR_TYPE` with the type of error to be stored in the `ERROR_TYPE` column in the `ELR_ERROR` table. The type is set to 'Rhapsody' because the error is an internal Rhapsody error.

Sets the property `PROCESSING_STEP` with the Rhapsody route name and filter name to be stored in the `PROCESSING_STEP` column in the `ELR_ERROR` table.

Sets the property `ERROR_MSG` with the Rhapsody generated errors to be stored in the `ERROR_MSG` column in the `ELR_ERROR` table.

Sets the property `ENVIRONMENT` with the Rhapsody Variable `ECLIR_ENVIRONMENT` to be stored in the `ENVIRONMENT` column in the `ELR_MESSAGE` table.

Sets the content of the message back to the original to be stored in the `ORIGINAL_MESSAGE` column in the `ELR_MESSAGE` table if needed.

Javascript

```
//
=====
// _____ INITIAL VARIABLES (start
)
//
=====

var next = output.append(input[0]);

//
=====
// _____ INITIAL VARIABLES (end
)
//
=====

//
=====
// _____ SET MESSAGE PROPERTIES FOR DB INSERT (start
)
//
=====

next.setProperty('VALIDATION_TYPE', 'Rhapsody');
next.setProperty('ERROR_CATEGORY', 'FATAL');
next.setProperty('ERROR_TYPE', 'Rhapsody');
next.setProperty('PROCESSING_STEP', 'Parsing: ECLIR - ELR 2.5.1 Message Parsing (Route) ->' +
  next.getProperty('rhapsody:ErrorFilterName') + ' (Filter)');
//next.setProperty('ERROR_MSG', 'Internal Rhapsody Route Error. Message processing could not
  continue: ' + next.getErrors());
var ERROR_MSG = 'Internal Rhapsody Route Error. Message processing could not continue: ' + next
  .getErrors();
next.setProperty('ERROR_MSG', ERROR_MSG);
```

Javascript

```
//Send the message to the Error Queue if it was caused by a database connectivity issue.
if (ERROR_MSG.indexOf('The connection is broken and recovery is not possible.') != -1 || ERROR_MSG == 'Internal Rhapsody Route Error. Message processing could not continue: ') {
    next.setProperty('SendToErrorQueue', true);
}

//This is a Rhapsody variable that should be set up during configuration.
var [REDACTED]_ENVIRONMENT = getVariable("ECLIR_[REDACTED]_ENVIRONMENT");
if ([REDACTED]_ENVIRONMENT == null || [REDACTED]_ENVIRONMENT == "") {
    [REDACTED]_ENVIRONMENT = "VARIABLE NOT SETUP IN RHAPSODY";
}
next.setProperty('[REDACTED]_ENVIRONMENT', [REDACTED]_ENVIRONMENT);

// Load the snapshot back into the message.
if (next.getProperty("SnapshotIDofOrigMsg") != null) {
    next.text = next.loadMessageSnapshotForId(next.getProperty("SnapshotIDofOrigMsg")).text;
}

//
=====
// _____ SET MESSAGE PROPERTIES FOR DB INSERT (end
//
=====
```

Destinations

Name	Page
Output Condition	
SendToErrorQueue = 'true' ignoring case	
🔔 Error Queue..... Error Queue	259
Error Connector	Page
🗄 db ELR Parsing Route Errors..... Database Lookup	261

**db Insert ELR parsed data into ECLIR**

Filter

Database Lookup

On ECLIR - ELR 2.5.1 Message Parsing route

Notes

Purpose: This Database Lookup filter loops through the XML flat file created in the *js Parse ELR to XML for DB Inserts* JavaScript filter. The parsed message data is written to the corresponding ELR tables in the ECLIR database.

Configuration Properties

Maximum Concurrency	1
Database	[REDACTED]

Configuration Properties		
Host	[REDACTED]	
Port	[REDACTED]	
Database Name	[REDACTED]	
Username	[REDACTED]	
Password	[REDACTED]	
Additional Connection Properties	Property Name	Property Value
	[REDACTED]	[REDACTED]
Configuration File	[REDACTED]	
Message Type	XML	
Definition		
Query Timeout		
Deadlock Retry Attempts	5	
Deadlock Retry Delay	100	
Socket Timeout	0	
Configuration File:	[REDACTED]	

```
<statement name="Insert Into ELR_MESSAGE Table" ide:type="executeSql" ide:summary="Insert
data into the ELR_MESSAGE table. One record per ELR message." ide:allowCompoundStatements
="true">
```

```
<sql>INSERT INTO ELR_MESSAGE (
ORIGINAL_MESSAGE_FORMAT
,ORIGINAL_MESSAGE
,BASE_FILENAME
,ENVIRONMENT
,SUCCESS_FL
,MSH_1
,MSH_2
,MSH_3_1
,MSH_3_2
,MSH_3_3
,MSH_4_1
,MSH_4_2
,MSH_4_3
,MSH_5_1
,MSH_5_2
,MSH_5_3
,MSH_6_1
,MSH_6_2
,MSH_6_3
,MSH_7
,MSH_7_CONVERTED
,MSH_9_1
,MSH_9_2
,MSH_9_3
,MSH_10
,MSH_11
,MSH_12_1
,MSH_15
```

Configuration File:

```

,MSH_16
,MSH_21_1_LIST
,INSERT_DATE
)
VALUES (
~/message/rowELR_MESSAGE/ORIGINAL_MESSAGE_FORMAT`
,$ssOriginalMsg
,LEFT($FullFilename,1000)
~/message/rowELR_MESSAGE/ENVIRONMENT`
~/message/rowELR_MESSAGE/SUCCESS_FL`
~/message/rowELR_MESSAGE/MSH_1`
~/message/rowELR_MESSAGE/MSH_2`
~/message/rowELR_MESSAGE/MSH_3_1`
~/message/rowELR_MESSAGE/MSH_3_2`
~/message/rowELR_MESSAGE/MSH_3_3`
~/message/rowELR_MESSAGE/MSH_4_1`
~/message/rowELR_MESSAGE/MSH_4_2`
~/message/rowELR_MESSAGE/MSH_4_3`
~/message/rowELR_MESSAGE/MSH_5_1`
~/message/rowELR_MESSAGE/MSH_5_2`
~/message/rowELR_MESSAGE/MSH_5_3`
~/message/rowELR_MESSAGE/MSH_6_1`
~/message/rowELR_MESSAGE/MSH_6_2`
~/message/rowELR_MESSAGE/MSH_6_3`
~/message/rowELR_MESSAGE/MSH_7`
~/message/rowELR_MESSAGE/MSH_7_CONVERTED`
~/message/rowELR_MESSAGE/MSH_9_1`
~/message/rowELR_MESSAGE/MSH_9_2`
~/message/rowELR_MESSAGE/MSH_9_3`
~/message/rowELR_MESSAGE/MSH_10`
~/message/rowELR_MESSAGE/MSH_11`
~/message/rowELR_MESSAGE/MSH_12_1`
~/message/rowELR_MESSAGE/MSH_15`
~/message/rowELR_MESSAGE/MSH_16`
~/message/rowELR_MESSAGE/MSH_21_1_LIST`
,SYSDATETIME()
)</sql>
<statement ide:type="compoundSqlStatement">
  <sql>SELECT \@@IDENTITY AS ELR_MESSAGE_ID</sql>
  <substituteColumn name="ELR_MESSAGE_ID">$ELR_MESSAGE_ID</substituteColumn>
</statement>
<statement name="Insert Into ELR_MESSAGE_SFT Table" ide:type="executeSql" iterate="SFT
_GROUP/rowELR_MESSAGE_SFT" ide:summary="Insert data into the ELR_MESSAGE_SFT table. One ELR
message to many SFT segments." ide:allowCompoundStatements="true" user:iterationRootMessage
Path="message">
  <sql>INSERT INTO ELR_MESSAGE_SFT (
    ELR_MESSAGE_ID,
    SFT_1,
    SFT_2,
    SFT_3,
    SFT_4,
    SFT_5,
    SFT_6,
    INSERT_DATE
  )
  VALUES (
    @ELR_MESSAGE_ID
    ,~/message/SFT_GROUP/rowELR_MESSAGE_SFT/SFT_1`

```

Configuration File:

```

,~/message/SFT_GROUP/rowELR_MESSAGE_SFT/SFT_2`
,~/message/SFT_GROUP/rowELR_MESSAGE_SFT/SFT_3`
,~/message/SFT_GROUP/rowELR_MESSAGE_SFT/SFT_4`
,~/message/SFT_GROUP/rowELR_MESSAGE_SFT/SFT_5`
,~/message/SFT_GROUP/rowELR_MESSAGE_SFT/SFT_6`
,SYSDATETIME()
) </sql>
  </statement>
  <statement name="PATIENT_RESULT_GROUP" ide:type="executeSql" iterate="PATIENT_RESULT_GROUP"
ide:allowCompoundStatements="true" user:iterationRootMessagePath="message">
    <statement name="Insert Into ELR_MESSAGE_PID Table" ide:type="executeSql" ide:summary
="Insert data into the ELR_MESSAGE_PID table. One PID segment to one PATIENT_RESULT group."
ide:allowCompoundStatements="true">
      <sql>INSERT INTO ELR_MESSAGE_PID (
ELR_MESSAGE_ID
,PID_1
,PID_3
,PID_3_1_RE1
,PID_3_5_RE1
,PID_3_1_RE2
,PID_3_5_RE2
,PID_3_1_RE3
,PID_3_5_RE3
,PID_5
,PID_6_1_1
,PID_7
,PID_7_CONVERTED
,PID_8
,PID_10
,PID_10_1_LIST
,PID_10_2_LIST
,PID_10_3_LIST
,PID_10_MULTI_FL
,PID_11
,PID_11_PRIMARY_STATE
,PID_11_PRIMARY_COUNTY
,PID_13
,PID_14
,PID_15
,PID_16
,PID_17
,PID_18_1
,PID_21_1_LIST
,PID_22
,PID_22_1_LIST
,PID_22_2_LIST
,PID_22_3_LIST
,PID_22_MULTI_FL
,PID_23
,PID_24
,PID_25
,PID_26
,PID_27
,PID_29
,PID_29_CONVERTED
,PID_30
,PID_31
,PID_32

```

Configuration File:

```

,PID_33
,PID_33_CONVERTED
,PID_34
,PID_39
,NTE_INCLUDED_FL
,PV1_1
,PV1_2
,PV1_4
,PV1_14
,PV1_44
,PV1_44_CONVERTED
,PV1_45
,PV1_45_CONVERTED
,INSERT_DATE
)
VALUES (
@ELR_MESSAGE_ID
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_1~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_3~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_3_1_RE1~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_3_5_RE1~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_3_1_RE2~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_3_5_RE2~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_3_1_RE3~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_3_5_RE3~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_5~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_6_1_1~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_7~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_7_CONVERTED~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_8~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_10~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_10_1_LIST~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_10_2_LIST~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_10_3_LIST~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_10_MULTI_FL~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_11~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_11_PRIMARY_STATE~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_11_PRIMARY_COUNTY~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_13~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_14~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_15~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_16~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_17~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_18_1~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_21_1_LIST~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_22~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_22_1_LIST~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_22_2_LIST~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_22_3_LIST~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_22_MULTI_FL~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_23~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_24~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_25~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_26~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_27~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_29~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_29_CONVERTED~
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_30~

```

Configuration File:

```

,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_31`
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_32`
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_33`
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_33_CONVERTED`
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_34`
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PID_39`
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/NTE_INCLUDED_FL`
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PV1_1`
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PV1_2`
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PV1_4`
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PV1_14`
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PV1_44`
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PV1_44_CONVERTED`
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PV1_45`
,~/message/PATIENT_RESULT_GROUP/PATIENT/rowELR_MESSAGE_PID/PV1_45_CONVERTED`
,SYSDATETIME()
) </sql>
<statement ide:type="compoundSqlStatement">
  <sql>SELECT \@\\@IDENTITY AS ELR_MESSAGE_PID_ID</sql>
</statement>
</statement>
<statement name="Insert Into ELR_MESSAGE_NK1 Table" ide:type="executeSql" iterate="PATIENT_RESULT_GROUP/PATIENT/NK1_GROUP/rowELR_MESSAGE_NK1" ide:summary="Insert data into the ELR_MESSAGE_NK1 table. Many NK1 segments to PATIENT group." ide:allowCompoundStatements="true" user:iterationRootMessagePath="message">
  <sql>INSERT INTO ELR_MESSAGE_NK1 (
    ELR_MESSAGE_ID
    ,ELR_MESSAGE_PID_ID
    ,NK1_1
    ,NK1_2
    ,NK1_3_1
    ,NK1_3_2
    ,NK1_3_3
    ,NK1_4
    ,NK1_5
    ,NK1_13
    ,NK1_20
    ,NK1_30
    ,NK1_31
    ,NK1_32
    ,INSERT_DATE
  )
  VALUES (
    @ELR_MESSAGE_ID
    ,@ELR_MESSAGE_PID_ID
    ,~/message/PATIENT_RESULT_GROUP/PATIENT/NK1_GROUP/rowELR_MESSAGE_NK1/NK1_1`
    ,~/message/PATIENT_RESULT_GROUP/PATIENT/NK1_GROUP/rowELR_MESSAGE_NK1/NK1_2`
    ,~/message/PATIENT_RESULT_GROUP/PATIENT/NK1_GROUP/rowELR_MESSAGE_NK1/NK1_3_1`
    ,~/message/PATIENT_RESULT_GROUP/PATIENT/NK1_GROUP/rowELR_MESSAGE_NK1/NK1_3_2`
    ,~/message/PATIENT_RESULT_GROUP/PATIENT/NK1_GROUP/rowELR_MESSAGE_NK1/NK1_3_3`
    ,~/message/PATIENT_RESULT_GROUP/PATIENT/NK1_GROUP/rowELR_MESSAGE_NK1/NK1_4`
    ,~/message/PATIENT_RESULT_GROUP/PATIENT/NK1_GROUP/rowELR_MESSAGE_NK1/NK1_5`
    ,~/message/PATIENT_RESULT_GROUP/PATIENT/NK1_GROUP/rowELR_MESSAGE_NK1/NK1_13`
    ,~/message/PATIENT_RESULT_GROUP/PATIENT/NK1_GROUP/rowELR_MESSAGE_NK1/NK1_20`
    ,~/message/PATIENT_RESULT_GROUP/PATIENT/NK1_GROUP/rowELR_MESSAGE_NK1/NK1_30`
    ,~/message/PATIENT_RESULT_GROUP/PATIENT/NK1_GROUP/rowELR_MESSAGE_NK1/NK1_31`
    ,~/message/PATIENT_RESULT_GROUP/PATIENT/NK1_GROUP/rowELR_MESSAGE_NK1/NK1_32`
    ,SYSDATETIME()
  )

```

Configuration File:

```

) </sql>
  </statement>
  <statement name="ORDER_OBSERVATION_GROUP" ide:type="executeSql" iterate="PATIENT_RESULT
_GROUP/ORDER_OBSERVATION_GROUP" ide:allowCompoundStatements="true" user:iterationRootMessage
Path="message">
    <statement name="Insert Into ELR_MESSAGE_OBR Table" ide:type="executeSql" ide:summary
="Insert data into the ELR_MESSAGE_OBR table. Many OBR segments to PATIENT_RESULT.ORDER_OBS
ERVATION group." ide:allowCompoundStatements="true">
      <sql>INSERT INTO ELR_MESSAGE_OBR (
ELR_MESSAGE_ID
,ELR_MESSAGE_PID_ID
,FILLER_ORDER_NUMBER
,ORDER_GROUP_ID
,ORC_1
,ORC_2_1
,ORC_2_2
,ORC_2_3
,ORC_2_4
,ORC_3_1
,ORC_3_2
,ORC_3_3
,ORC_3_4
,ORC_4_1
,ORC_5
,ORC_6
,ORC_8
,ORC_9
,ORC_10
,ORC_11
,ORC_12
,ORC_13
,ORC_14
,ORC_15
,ORC_16
,ORC_17
,ORC_18
,ORC_19
,ORC_21
,ORC_22
,ORC_23
,ORC_24
,ORC_25
,ORC_27
,ORC_28
,ORC_29
,ORC_30
,ORC_31
,OBR_1
,OBR_2_1
,OBR_2_2
,OBR_2_3
,OBR_2_4
,OBR_3_1
,OBR_3_2
,OBR_3_3
,OBR_3_4
,OBR_4_1
,OBR_4_2

```